How-To: The Connector System

ragent can reach external systems through **connectors**: named, user-facing integrations (Google Drive, Slack, GitHub, Git, Postgres, Puppeteer, and the wider community catalogue) that connect through one or more **MCP servers**. A connector is a catalogue entry carrying a display name, a category, an authentication requirement, and one or more MCP servers; the MCP server remains the transport ragent already speaks, so the connector system is a management and catalogue layer over machinery that already exists.

This document is the practical manual: what a connector is, how to browse, install, authenticate, and manage one, and how the configuration works.

Table of Contents

- 1. Overview
- 2. What a Connector Is
- 3. The Connector Store
- 4. Browsing the Catalogue
- 5. Installing, Enabling, and Managing
- 6. Authentication
- 7. Testing a Connector
- 8. Configuration Reference
- 9. Troubleshooting
- 10. Reference

---

1. Overview

ragent already has the transport (`McpClient`), a durable enable/disable ledger (`mcp_state.json`), an MCP discovery scanner, and a plugin-store browser. What the connector system adds is the **connector abstraction**: a catalogue entry that names an integration, points at one or more MCP servers, and carries its authentication shape, managed through one coherent surface (`/connectors`).

If you have used Claude's connectors, this is the equivalent: pick an integration from a catalogue, authenticate it, and its tools appear in the session.

2. What a Connector Is

A connector is a normalised descriptor with:

- an **id** and a **display name**;
- a **category** (for example `productivity`, `developer`, `data`);
- one or more **MCP server definitions** (stdio, SSE, or HTTP);
- an optional **authentication shape** and the **name** of a credential held in the encrypted credential store.

Because every connector resolves to one or more MCP servers, its tools surface to the model under the normal `mcp_<sanitized-server>_<tool>` names, and the permission system treats them exactly as any other MCP tool.

Bridged server ids are `<connector-id>.<server>` (the same collision-avoidance shape the plugin MCP bridge uses).

3. The Connector Store

Connectors are discovered under:

- `.ragent/connectors/` (project; highest priority), or
- `~/.config/ragent/connectors/` (user-global),

with an optional `connectors.store_dir` override. A freshly installed connector is recorded **enabled**; run `/connectors enable <id>` to connect it now in a running session.

The durable enable state is the existing global `mcp_state.json` ledger shared with the rest of MCP, so a connector disabled once stays disabled everywhere and survives a restart.

4. Browsing the Catalogue

`/connectors claude [query] [--category <name>] [--refresh]` opens an interactive catalogue browser:

- a title line with the catalogue name, the current query, the active category, and the visible/total result count; a `loading details...` hint is appended while the per-entry manifest description enrichment is still running;
- a fixed column-heading row (`Id`, `Category`, `Marketplace`, `Description`) pinned directly above the results and painted dark-on-light; it never scrolls with the rows and a click or wheel on it is inert;
- a scrollable list of results, sorted alphabetically by connector id (case-insensitive), with the block cursor on the highlighted row; each row shows the connector id, its category, the marketplace it came from (or `-` when it came from a local install), and its description; a category filter narrows the already-sorted list, so the visible rows always stay alphabetical;
- an `[installed]` marker and distinct colour on connectors you already have;
- a footer of key hints.

An optional `query` pre-fills the search field and `--category <name>` pre-selects the category filter; a category the catalogue does not declare is refused in the panel footer and no row is hidden. `--refresh` bypasses the catalogue cache.

The panel fetches the compiled default catalogue endpoint and every marketplace added with `/connectors marketplace claude add`, merging their entries into the one list (a duplicate id keeps the first occurrence, the base endpoint's) and naming each row's marketplace. A refused added marketplaces URL is skipped and reported as a warning in the message window rather than blinding the panel.

While the panel is open it owns the keyboard: `Up`/`Down` move the cursor, `ENTER` installs the highlighted result, `Backspace` and `Esc` edit the query (and `Esc` on an empty query dismisses the panel). A left-hand category navigator lists `ALL` plus the categories the catalogue declares: `Tab` moves focus between it and the result list, `Up`/`Down` over the navigator apply the highlighted category immediately, `ENTER` over the navigator applies and hands focus back to the results, and `c` clears the filter back to `ALL`. The mouse works too: click a category row to apply it, wheel over the navigator to scroll it. The catalogue fetch runs off the event loop, so the UI keeps animating while it loads.

`/connectors marketplace show` lists each marketplace configured for the Claude catalogue (the base endpoint plus every added marketplace, tagged `default`/`config`/`added`); `/connectors marketplace claude add <source> [--force]` and `/connectors marketplace claude remove <name>` record and drop added marketplaces in your user-global config.

`/connectors stores [--check]` reports each catalogue endpoint and its provenance (`default` or `config`).

5. Installing, Enabling, and Managing

Step	Command
Install	`/connectors add <id|source>` (recorded enabled)
Enable and connect now	`/connectors enable <id>`
Disconnect but keep enabled	`/connectors disconnect <id>`
Reconnect	`/connectors connect <id>`
Disable and disconnect	`/connectors disable <id>`
Uninstall	`/connectors remove <id>` (refused while enabled)
List	`/connectors list [--verbose] [--category <name>]`

`/connectors add` accepts a catalogue connector id, a local directory holding a connector manifest, a local `.zip`/`.tar.gz`, or an `https://` URL naming a package. A duplicate id is refused unless `--force` is given; an archive member that would escape the store root is refused; a non-`https` package URL is refused.

`/connectors list --category <name>` restricts the rows to one category; `ALL` (any case) or a blank value clears the filter, and an unknown category is refused with an `[err]` row and changes no state. The same `--category` filter is accepted by `/connectors claude` to open the browser with that category already selected.

6. Authentication

A connector's secrets (tokens, client secrets, refresh tokens) live in the existing **encrypted credential store**; the connector manifest stores only the credential *name*, never the value.

`/connectors auth <id>` reports a connector's auth state and lets a credential be supplied. A connector that requires authentication and has no valid credential reports `needs auth`, and its servers are refused until it is satisfied (there is no retry loop on an auth failure).

7. Testing a Connector

`/connectors test <id>` connects the connector's servers **in isolation** with a throwaway probe, invokes one tool, and reports per-step `[ ok ]`/`[fail]` results. It never touches the live session, so it is safe to run against a connector you are unsure about.

8. Configuration Reference

The `connectors` block in `ragent.json` controls the subsystem:

```
{
  "connectors": {
    "enabled": true,                        // master switch; default true
    "store_dir": null,                      // optional store-path override
    "stores": {
      "community": { "url": "https://example.org/connectors/index.json" },
```

```
      "timeout_ms": 10000,
      "max_index_bytes": 2097152,
      "cache_ttl_secs": 3600
    },
    "credentials": {
      "google-drive": { "token": "GDRIVE_TOKEN" }
    }
  }
}
```

The section merges overlay-wins (project config overrides user-global). With `connectors.enabled: false` the subsystem performs no discovery, no catalogue fetch, and no connection, and every subcommand other than `help` reports that it is disabled. See `docs/howtos/config.md` §7.37 for the full field table.

9. Troubleshooting

- **`the connector subsystem is disabled`** - set `connectors.enabled: true`.
- **A connector lists as `needs auth`** - run `/connectors auth <id>` and supply the credential.
- **`remove` is refused** - disable the connector first with `/connectors disable <id>`.
- **A catalogue fetch fails** - check `/connectors stores --check` for the endpoint and its provenance, and confirm network access to the `https` URL.
- **An unexpressible entry is skipped** - a catalogue entry whose transport cannot be expressed as an MCP server (for example `grpc`) is skipped and counted, not fatal.

10. Reference

- `docs/howtos/slashcommands/connectors.md` - the `/connectors` slash-command reference
- `docs/howtos/config.md` §7.37 - the `connectors` config block
- `docs/howtos/mcp.md` - MCP server status, discovery, and enable/disable
- `docs/howtos/plugins.md` - the sandboxed plugin system (a sibling catalogue)
- `ragent connectors <sub>` - CLI parity for the same subcommands
- `specs/connectors/SPEC.md` - the full connector-system specification